

Package ‘excheckr’

February 13, 2026

Type Package

Title Tools for Exploring and Checking Experimental Data

Version 0.1.0

Description Functions for diagnosing, summarizing, and visualizing missingness and data integrity in experimental datasets. Includes tools for identifying missing covariates and producing visual summaries of joint missingness.

License MIT + file LICENSE

Encoding UTF-8

RoxygenNote 7.3.3

Imports dplyr, tidyselect, rlang, purrr, glue, tibble, ggplot2, broom

Suggests estimatr, knitr, rmarkdown, testthat (>= 3.0.0)

Config/testthat/edition 3

VignetteBuilder knitr

NeedsCompilation no

Author Jane Doe [aut, cre]

Maintainer Jane Doe <jane@example.com>

Depends R (>= 4.1.0)

Contents

check_attrition	2
check_balance	3
check_covariate_missingness	5
hello	6
stat_mode	6
write_attrition_check_code	7
write_balance_check_code	8
write_covariate_imputation_code	9
write_outcome_missingness_dummies_code	10

Index	11
--------------	-----------

check_attrition	<i>Check differential attrition across treatment conditions</i>
-----------------	---

Description

Performs attrition checks by regressing outcome missingness indicators on treatment assignment to test for differential attrition.

Usage

```
check_attrition(
  data,
  treatment,
  outcomes = NULL,
  covariates = NULL,
  .method = estimatr::lm_robust,
  ...
)
```

Arguments

data	A data frame or tibble.
treatment	Unquoted name of the treatment variable.
outcomes	Character vector of outcome variable names, or unquoted column names using tidyselect helpers. If left empty, all <code>"Y_"</code> columns are used.
covariates	Character vector of covariate names, or unquoted column names using tidyselect helpers. If provided, fits a Lin (2013) model interacting treatment with demeaned covariates and performs an F-test comparing the full model (<code>missingness ~ treatment * covariates</code>) to the restricted model (<code>missingness ~ covariates</code>).
.method	Regression function to use (default: <code>'estimatr::lm_robust'</code>). Must accept formula and data arguments.
...	Additional arguments passed to <code>'method'</code> (e.g., <code>'clusters'</code> , <code>'se_type'</code>).

Details

For each outcome variable, creates a missingness indicator (1 if missing, 0 otherwise) and regresses it on treatment assignment. Significant coefficients indicate differential attrition across treatment conditions.

If missingness indicator variables already exist (with `'_missing'` suffix), those are used. Otherwise, they are created on the fly.

When covariates are provided, the function follows the Lin (2013) estimator approach used in `estimatr::lm_lin`: covariates are demeaned by subtracting the full-sample mean, then the full model `missingness ~ treatment * (demeaned covariates)` is fit. A Wald F-test compares this to the restricted model `missingness ~ demeaned covariates`, testing whether treatment and its interactions with covariates jointly predict attrition.

Value

When `covariates` is `NULL`, a tibble with one row per outcome containing the treatment coefficient from regressing missingness on treatment. When `covariates` is provided, a list with two elements:

coefficients A tibble of all coefficient estimates from the Lin model (treatment, demeaned covariates, and their interactions).

f_test A tibble with one row per outcome containing the Wald F-test of joint significance of treatment and treatment-by-covariate interactions.

Examples

```
## Not run:
library(estimatr)

dat <- data.frame(
  Z = rbinom(200, 1, 0.5),
  X_age = rnorm(200, 50, 10),
  X_income = rnorm(200, 50000, 10000),
  Y_attitude = c(rnorm(150), rep(NA, 50)),
  Y_behavior = c(rnorm(180), rep(NA, 20))
)

# Simple attrition check (no covariates)
check_attrition(dat, Z)

# With covariates: Lin model + F-test for differential attrition
check_attrition(dat, Z, covariates = c("X_age", "X_income"))

# With clustered standard errors
dat$cluster_id <- sample(1:10, 200, replace = TRUE)
check_attrition(dat, Z, clusters = cluster_id)

## End(Not run)
```

check_balance

Check covariate balance across treatment conditions

Description

Performs balance checks by regressing each covariate on treatment assignment (covariate-by-covariate F-tests) and a joint test of all covariates together. Supports both binary and multi-armed treatments.

Usage

```
check_balance(
  data,
  treatment,
  covariates = NULL,
  .method = estimatr::lm_robust,
  ...
)
```

Arguments

<code>data</code>	A data frame or tibble.
<code>treatment</code>	Unquoted name of the treatment variable.
<code>covariates</code>	Character vector of covariate names, or unquoted column names using tidyselect helpers. If left empty, all <code>"X_"</code> columns are used.
<code>.method</code>	Regression function to use (default: <code>'estimatr::lm_robust'</code>). Must accept formula and data arguments.
<code>...</code>	Additional arguments passed to <code>'method'</code> (e.g., <code>'clusters'</code> , <code>'se_type'</code>).

Details

For numeric covariates, regresses the covariate on treatment directly and extracts the overall model F-test (which jointly tests all treatment dummies for multi-armed designs). For factor/character covariates, creates dummy variables for each level (excluding the first level as reference) and regresses each dummy on treatment.

The joint test regresses treatment on all covariates simultaneously. For binary treatment, this is a single regression with an F-test. For multi-armed treatment (3+ levels), this uses a cross-equation Wald test across K-1 linear probability models with a cluster-robust sandwich estimator.

Value

A list with two elements:

covariate_tests A tibble with one row per covariate (or covariate level for factors), containing the F-test from regressing each covariate on treatment. Columns: `covariate`, `level`, `F_stat`, `df1`, `df2`, `p_value`, `nobs`.

joint_test A tibble with a single row containing the joint F-test of all covariates predicting treatment. Columns: `F_stat`, `df1`, `df2`, `p_value`, `nobs`.

Examples

```
## Not run:
library(estimatr)

dat <- data.frame(
  Z = rbinom(100, 1, 0.5),
  X_age = rnorm(100, 50, 10),
  X_gender = sample(c("M", "F"), 100, replace = TRUE),
  X_party = factor(sample(c("D", "R", "I"), 100, replace = TRUE))
)

# Default: all X_ covariates with lm_robust
check_balance(dat, Z)

# Specific covariates
check_balance(dat, Z, c("X_age", "X_gender"))

# With clustered standard errors
dat$cluster_id <- sample(1:10, 100, replace = TRUE)
check_balance(dat, Z, clusters = cluster_id)

# Multi-armed treatment
dat2 <- data.frame(
```

```

    Z = factor(sample(c("C", "T1", "T2"), 100, replace = TRUE)),
    X_age = rnorm(100), X_income = rnorm(100)
  )
  check_balance(dat2, Z)

## End(Not run)

```

check_covariate_missingness

Summarize and visualize missingness of covariates

Description

Computes missingness summaries for a set of covariates and displays a joint missingness heatmap (upper-triangle including the diagonal).

Usage

```
check_covariate_missingness(data, ...)
```

Arguments

<code>data</code>	A data frame or tibble containing the covariates.
<code>...</code>	Columns to include in the analysis. You can specify them unquoted (e.g., <code>'age'</code> , <code>'income'</code>) or using selection helpers such as <code>[dplyr::all_of()]</code> or <code>[tidyselect::starts_with()]</code> . If left empty, all <code>"X_"</code> columns are used.

Details

If no variables are specified, it defaults to all variables starting with `"X_"`, excluding columns ending with `'_nona'` or `'_missing'`.

This function supports both unquoted column names and tidyselect helpers such as `[dplyr::all_of()]`, `[tidyselect::starts_with()]`, etc.

Value

Invisibly returns a list with two elements:

summary A tibble summarizing total cases, total missing, and fraction missing per variable.

heatmap A `[ggplot2::ggplot()]` object showing the upper-triangle joint missingness (including diagonal).

Examples

```

dat <- data.frame(
  X_pid_3 = c("A", NA, "B", "A", NA, "B", "A", NA, "B", "A"),
  X_income = c(100, 200, NA, 400, NA, 300, 500, NA, 600, 700),
  X_age = c(25, NA, 30, 40, NA, 35, 45, NA, 50, 55)
)

# X_pid_3 and X_income have similar missingness

```

```

check_covariate_missingness(dat, X_pid_3, X_income, X_age)

# Or default to all "X_" columns
check_covariate_missingness(dat)

# Or use tidyselect helpers
vars <- c("X_pid_3", "X_income", "X_age")
check_covariate_missingness(dat, dplyr::all_of(vars))

```

hello	<i>Hello, World!</i>
-------	----------------------

Description

Prints 'Hello, world!'.

Usage

```
hello()
```

Examples

```
hello()
```

stat_mode	<i>Statistical mode</i>
-----------	-------------------------

Description

Computes the most frequent (modal) value of a vector.

Usage

```
stat_mode(x, na.rm = TRUE)
```

Arguments

x	A vector (numeric, character, or factor).
na.rm	Logical. Should missing values be removed before computing the mode? Defaults to TRUE.

Value

The most frequent value of 'x'. If 'x' is a factor, the result is returned as a factor with the same levels. If there are ties, the first occurring mode is returned. If all values are missing, returns 'NA'.

Examples

```

stat_mode(c(1, 2, 2, 3, NA))
stat_mode(c("a", "b", "a", "c", "c"))
stat_mode(factor(c("low", "high", "low", NA)))

```

`write_attrition_check_code`*Write attrition check code*

Description

Generates code to perform attrition checks by regressing outcome missingness indicators on treatment assignment.

Usage

```
write_attrition_check_code(  
  data,  
  treatment,  
  outcomes = NULL,  
  .method = estimatr::lm_robust,  
  ...  
)
```

Arguments

<code>data</code>	A data frame or tibble.
<code>treatment</code>	Unquoted name of the treatment variable.
<code>outcomes</code>	Character vector of outcome variable names, or unquoted column names using tidyselect helpers. If left empty, all <code>"Y_"</code> columns are used.
<code>.method</code>	Regression function to use (default: <code>'estimatr::lm_robust'</code>).
<code>...</code>	Additional arguments passed to <code>'method'</code> (e.g., <code>'clusters'</code> , <code>'se_type'</code>).

Details

This function prints R code to the console that you can copy-paste into your analysis script. It does not perform the attrition check itself.

Value

Invisibly returns the generated code as a single string.

Examples

```
## Not run:  
dat <- data.frame(  
  Z = rbinom(200, 1, 0.5),  
  Y_attitude = c(rnorm(150), rep(NA, 50)),  
  Y_behavior = c(rnorm(180), rep(NA, 20))  
)  
  
write_attrition_check_code(dat, Z)  
  
## End(Not run)
```

```
write_balance_check_code
```

Write balance check code

Description

Generates code to perform balance checks by regressing each covariate on treatment assignment.

Usage

```
write_balance_check_code(
  data,
  treatment,
  covariates = NULL,
  .method = estimatr::lm_robust,
  ...
)
```

Arguments

<code>data</code>	A data frame or tibble.
<code>treatment</code>	Unquoted name of the treatment variable.
<code>covariates</code>	Character vector of covariate names, or unquoted column names using tidyselect helpers. If left empty, all "X_" columns are used.
<code>.method</code>	Regression function to use (default: 'estimatr::lm_robust').
<code>...</code>	Additional arguments passed to '.method' (e.g., 'clusters', 'se_type').

Details

This function prints R code to the console that you can copy-paste into your analysis script. It does not perform the balance check itself. The joint balance test is generated as a call to `check_balance()`, since the cross-equation Wald test is too complex to emit as copy-paste code.

Value

Invisibly returns the generated code as a single string.

Examples

```
## Not run:
dat <- data.frame(
  Z = rbinom(100, 1, 0.5),
  X_age = rnorm(100, 50, 10),
  X_gender = sample(c("M", "F"), 100, replace = TRUE)
)

write_balance_check_code(dat, Z)

## End(Not run)
```

`write_covariate_imputation_code`*Write covariate imputation code*

Description

Generates tidyverse-style `'mutate()'` code to impute missing values for selected variables. Numeric variables are imputed with the median, while factor or character variables are imputed with the mode (user-defined). In addition to the imputed variable (with suffix `'_nona'`), a missingness dummy variable (with suffix `'_missing'`) is created for each input.

Usage

```
write_covariate_imputation_code(data, ..., include_missingness_dummies = TRUE)
```

Arguments

<code>data</code>	A data frame or tibble.
<code>...</code>	Columns to generate imputation code for. You can specify them unquoted (e.g., <code>'age'</code> , <code>'income'</code>) or using selection helpers such as <code>[dplyr::all_of()]</code> or <code>[tidyselect::starts_with()]</code> . If left empty, all <code>"X_"</code> columns are used.
<code>include_missingness_dummies</code>	Logical. Should missingness dummy variables be included in the generated code? Defaults to TRUE.

Details

This function prints imputation code to the console that you can copy-paste into your analysis script. It does not perform the imputation itself.

Value

Invisibly returns the generated code as a single string.

Examples

```
# Example data with missingness
dat <- data.frame(
  X_factor_variable = factor(rep(c("A", "B", NA), c(4, 5, 1))),
  X_numeric_variable = c(100, 200, NA, 400, 500, NA, 700, 800, 900, NA)
)

# Generate imputation code
write_covariate_imputation_code(dat, X_factor_variable, X_numeric_variable)

# Or default to all "X_" columns
write_covariate_imputation_code(dat)

# Or use tidyselect helpers
vars <- c("X_factor_variable", "X_numeric_variable")
write_covariate_imputation_code(dat, dplyr::all_of(vars))
```

`write_outcome_missingness_dummies_code`*Write outcome missingness code*

Description

Generates tidyverse-style `'mutate()'` code to generate missingness dummy variables (with suffix `'_missing'`) for selected variables.

Usage

```
write_outcome_missingness_dummies_code(data, ...)
```

Arguments

<code>data</code>	A data frame or tibble.
<code>...</code>	Columns to generate missingness dummy variables for. You can specify them unquoted (e.g., <code>'age'</code> , <code>'income'</code>) or using selection helpers such as <code>[dplyr::all_of()]</code> or <code>[tidyselect::starts_with()]</code> . If left empty, all <code>"Y_"</code> columns are used.

Details

This function prints mutate code to the console that you can copy-paste into your cleaning script. It does not create the variables itself.

Value

Invisibly returns the generated code as a single string.

Examples

```
# Example data with missingness
dat <- data.frame(
  Y_attitude = rep(c(1, 2, 3, 4, 5, NA), c(10, 20, 30, 40, 50, 50)),
  Y_behavior = rep(c(0, 1, NA), c(100, 50, 50))
)

# Generate missingness dummy code
write_outcome_missingness_dummies_code(dat, Y_attitude, Y_behavior)

# Or default to all "Y_" columns
write_outcome_missingness_dummies_code(dat)

# Or use tidyselect helpers
vars <- c("Y_attitude", "Y_behavior")
write_outcome_missingness_dummies_code(dat, dplyr::all_of(vars))
```

Index

check_attrition, [2](#)
check_balance, [3](#)
check_covariate_missingness, [5](#)

hello, [6](#)

stat_mode, [6](#)

write_attrition_check_code, [7](#)
write_balance_check_code, [8](#)
write_covariate_imputation_code, [9](#)
write_outcome_missingness_dummies_code,
[10](#)